

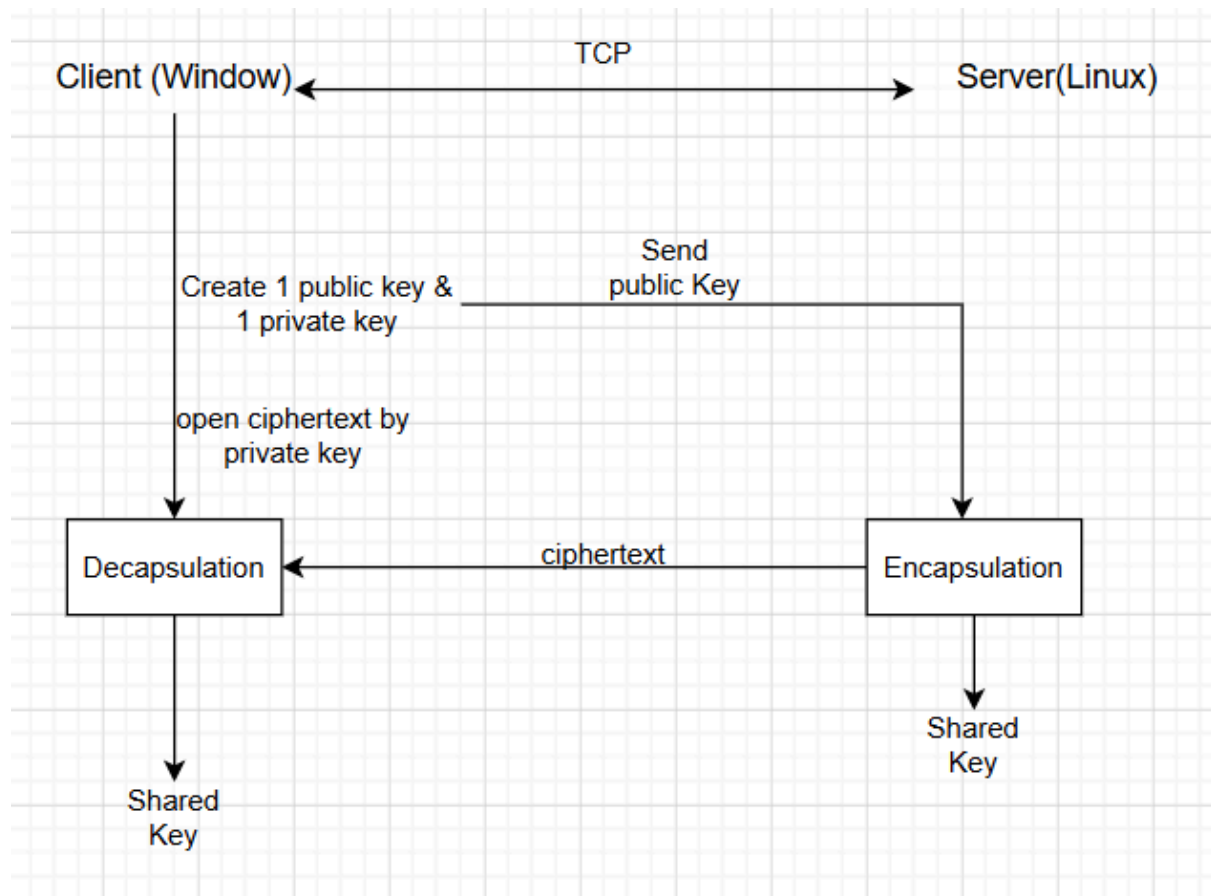
Mã hóa ML-KEM

I. Cơ sở lý thuyết:

1 Khái niệm:

-ML-KEM, trước đây được gọi là CRYSTALS-Kyber trước khi được NIST chuẩn hóa, là một Cơ chế Mã hóa Khóa (KEM) hậu lượng tử dựa trên MLWE. KEM là một cơ chế mật mã bất đối xứng cho phép tạo ra và truyền tải bí mật một cách an toàn bằng cách sử dụng một cặp khóa riêng tư-công khai. Nó bao gồm ba bước:

- Tạo khóa : một khóa công khai và một khóa riêng tư được tạo ra;
- Mã hóa : một bí mật được tạo ra và mã hóa bằng khóa công khai để tạo ra bản mã;
- Giải mã : văn bản mã hóa được giải mã bằng khóa riêng để khôi phục lại bí mật gốc.



2. Chi tiết các bước thực hiện

-Sẽ có 2 tham số cố định là $q=3329$ (các hệ số trong đa thức nhỏ hơn 3329)
, $n=256$ (số phần tử trong đa thức), k : kích thước ma trận , n_1, n_2

Bên client

-Sử dụng thuật toán Kyber-PKE để chọn ma trận A kích thước $k \times k$, đa thức s có hệ số khoảng $[-n_1, n_1]$, đa thức e có hệ số khoảng $[-n_2, n_2]$

-Tính $t = A \cdot s + e$

-Public key là (A, t) private key là s

Bên server:

-Nhận được (A, t)

-Chọn các đa thức r, e_1, e_2

-Tính $u = A^T \cdot r + e_1$ và $v = t^T \cdot r + e_2 + [q/2] \cdot m$

-Output ciphertext $c=(u, v)$

II. Mô phỏng bằng python:

1. Bên client

```

1  import socket
2  import pickle
3  import struct
4  from pqcrypto.kem.ml_kem_512 import encrypt
5
6  SERVER_IP = '192.168.1.6'
7  PORT = 5000
8
9  def send_packet(sock, data): 2 usages
10     raw = pickle.dumps(data)
11     sock.sendall(struct.pack('!I', *v: len(raw))) # gửi độ d
12     sock.sendall(raw)
13
14     client = socket.socket()
15     client.connect((SERVER_IP, PORT))
16
17     # Nhận public key từ server
18     data = client.recv(4096)
19     pk = pickle.loads(data)
20
21     # ML-KEM: tạo ciphertext và shared key
22     ciphertext, shared_key = encrypt(pk)
23
24     # Gửi ciphertext cho server
25     send_packet(client, ciphertext)
26
27     # Gửi plaintext trực tiếp (KHÔNG mã hóa)
28     plaintext = b"Hello FPGA"
29     send_packet(client, plaintext)
30
31     # Debug
32     print("Shared key (client):", shared_key.hex())
33     print(f"Public Key (hex): {pk.hex()[:50]}...")
34     print(f"ML-KEM Ciphertext: {ciphertext.hex()[:50]}...")
35     print(f"Plaintext sent: {plaintext}")
36
37     client.close()

```

-Giải thích đoạn code:

1. Thiết lập kết nối (Socket & Protocol)

- `send_packet`: Đây là hàm "đóng gói". Trong lập trình mạng, khi gửi dữ liệu qua TCP, các gói tin có thể bị dính vào nhau. Hàm này giải quyết bằng cách:
 1. Dùng `struct.pack("!I", len(raw))` để gửi 4 byte đầu tiên chứa độ dài của dữ liệu.
 2. Sau đó mới gửi nội dung thực (dùng `pickle` để biến đổi tượng Python thành chuỗi byte).
- `client.connect`: Thiết lập kết nối tới Server (địa chỉ IP và Cổng).
- `pk = pickle.loads(data)`: Client nhận Khóa công khai (Public Key) từ Server. Đây là bước khởi đầu của quá trình ML-KEM.

2. Giai đoạn ML-KEM (Trao đổi khóa)

`ciphertext, shared_key = encrypt(pk)`

- `encrypt(pk)`: Hàm này thực hiện bước Đóng gói (Encapsulation)
- `shared_key`: Đây là bí mật chung mà chỉ Client và Server biết. Client giữ lại giá trị này.
- `ciphertext`: Đây là "bản mã hóa" của khóa chung đó. Client gửi cái này cho Server qua lệnh `send_packet(client, ciphertext)`.
- Cơ chế: Server sẽ dùng Khóa bí mật `sk` của mình để giải mã `ciphertext` này nhằm lấy được `sharedkey` y hệt như Client.

Output in ra:

```
D:\PycharmProjects\ML_KEM\.venv\Scripts\python.exe D:\PycharmProjects\ML_KEM\client.py
Shared key (client): 9c04fd93a7408325e209229f5ac5bc0301eb2d4d76ced4e7331f5476f3572cf6
Public Key (hex): a0a2a7e4679792887939204ec7485ce4ac22c12759ba20625d...
ML-KEM Ciphertext: 88b8eb6ef5c3259f3193f0894bdc4eae89c589501a688eed00...
Plaintext sent: b'Hello FPGA'

Process finished with exit code 0
```

2. Bên server:

```

5  PORT = 5000
6
7  import struct
8
9  def recv_packet(sock): 1 usage
10     # nhận 4 byte độ dài
11     raw_len = sock.recv(4)
12     if not raw_len:
13         return None
14     length = struct.unpack(format: "!I", raw_len)[0]
15     data = b''
16     while len(data) < length:
17         packet = sock.recv(length - len(data))
18         if not packet:
19             break
20         data += packet
21
22     return pickle.loads(data)
23 pk, sk = generate_keypair()
24
25 server = socket.socket()
26 server.bind((HOST, PORT))
27 server.listen(1)
28
29 print("Server listening...")
30
31 conn, addr = server.accept()
32 print("Connected:", addr)
33 conn.sendall(pickle.dumps(pk))
34
35
36 ciphertext = recv_packet(conn)
37 shared_key = decrypt(sk, ciphertext)
38 print("Shared key (server):", shared_key.hex())
39
40
41 conn.close()
42 server.close()

```

-Giải thích đoạn code:

1. Khởi tạo khóa (Key Generation)

Đây là bước đầu tiên và quan trọng nhất của Server:

- pk (Public Key): Khóa công khai. Server sẽ gửi cái này cho bất kỳ ai (Client) muốn kết nối.
- sk (Secret Key): Khóa bí mật. Server giữ kỹ khóa này trong bộ nhớ và không bao giờ gửi đi. Đây là chìa khóa duy nhất có thể mở được các "bản mã" gửi từ Client.

2. Thiết lập kết nối Socket

```
server.bind((HOST, PORT))
```

```
server.listen(1)
```

```
conn, addr = server.accept()
```

- Server mở một "cửa sổ" tại cổng 5000 và đợi.
- Khi Client kết nối (accept), một đường truyền (conn) được thiết lập giữa hai bên.

3. Gửi Khóa công khai

```
conn.sendall(pickle.dumps(pk))
```

Server đóng gói và gửi pk cho Client. Như chúng ta đã biết ở các câu trước, Client sẽ dùng pk này để tạo ra một bí mật chung.

4. Cơ chế nhận gói tin (recv_packet)

Hàm này được viết để xử lý hiện tượng dính gói trong TCP:

1. Nhận 4 byte đầu tiên: Giải mã bằng struct.unpack để biết Client định gửi bao nhiêu dữ liệu (độ dài).
2. Vòng lặp while: Đảm bảo nhận đủ số byte đã báo trước đó thì mới dừng lại.
3. pickle.loads: Chuyển chuỗi byte nhận được trở lại thành đối tượng Python (ở đây là bản mã ciphertext).

5. Giải gói khóa (Decapsulation) - Trái tim của ML-KEM

```
ciphertext = recv_packet(conn)
shared_key = decrypt(sk, ciphertext)
```

- Server nhận ciphertext (Bản mã ciphertext dài 768 bytes) từ Client.
- Server dùng sk (Khóa bí mật) của mình để giải mã.
- Kết quả: Hàm decrypt trả về shared_key.

Output in ra:

```

| :
/home/hieu/PyCharmMiscProject/.venv/bin/python /home/hieu/PyCharmMiscProject/server.py
Server listening...
Connected: ('192.168.1.4', 50739)
Shared key (server): 9c04fd93a7408325e209229f5ac5bc0301eb2d4d76ced4e7331f5476f3572cf6

Process finished with exit code 0
```